Technical Code & Prompt Appendix

(Artificial Intelligence with Prompt Engineering) (DI05016011)

[Information Technology, Semester-V (Division - A)]

Enrolment No	**246230316006**
Name	**Aayush Bankar**
Branch	**Information Technology**
Repository Scope	**23 Source Files (Standalone + Privata RAG Engine)**
Institute	**Govt. Polytechnic Gandhinagar**



Directorate Of Technical Education Gandhinagar - Gujarat

Code & Prompt Files Index

Enrolment No: 246230316006 **Name: Aayush Bankar** **Total Files: 23**

#	Associated Practical	File Path in Repository	Description	Page
01	Practical 01	`p01_genai_task_templates.txt`	Generative AI Task Templates	3
02	Practical 02	`p02_sentiment_analysis.py`	Sentiment Analysis & Text Classification	4
03	Practical 02	`p02_lexicon_fallback.py`	Offline Lexicon Classifier Fallback	7
04	Practical 03	`p03_llm_behavior_test_matrix.md`	LLM Behavior Test Matrix	8
05	Practical 04	`p04_question_bank_and_rubric.md`	Question Bank & Evaluation Rubric	9
06	Practical 05	`p05_before_after_template.md`	Prompt Refinement Before/After Template	10
07	Practical 06	`p06_prompting_technique_templates.md`	Prompting Technique Templates	11
08	Practical 07	`p07_cot_and_chaining_templates.md`	Chain-of-Thought & Prompt Chaining	12
09	Practical 08	`p08_optimization_checklists.md`	Task Optimization Checklists	13
10	Practical 09	`p09_code_gen_debugging_cases.py`	AI Code Generation & Debugging Cases	14
11	Practical 10	`p10_chatbot.py`	AI Chatbot API Client with Mock Mode	16
12	Practical 11	`p11_rag_document_qa.py`	Document QA RAG Prototype	18
13	Practical 11	`p11_sample_document.txt`	Sample Knowledge Base Document	21
14	Practical 11 (Privata)	`privata/config.py`	Privata Central Configuration	22
15	Practical 11 (Privata)	`privata/ingest.py`	Privata Ingestion Pipeline	23
16	Practical 11 (Privata)	`privata/chat.py`	Privata Interactive Chat Loop	24
17	Practical 11 (Privata)	`privata/models/llm_loader.py`	Privata Gemini LLM Loader	25
18	Practical 11 (Privata)	`privata/retriever/embedder.py`	Privata Local Embedder	26
19	Practical 11 (Privata)	`privata/retriever/vectordb.py`	Privata ChromaDB Interface	27
20	Practical 11 (Privata)	`privata/utils/doc_loader.py`	Privata Document Loaders	28
21	Practical 11 (Privata)	`privata/requirements.txt`	Privata Environment Requirements	29
22	Practical 12	`p12_security_research_assistant.py`	Security Vulnerability & Mitigation Assistant	30
23	Practical 12	`p12_study_assistant.py`	AI Study Assistant Prototype	33

PRACTICAL 01 · FILE 01 OF 23 **FORMAT: TEXT · LENGTH: 67 LINES**

Generative AI Task Templates (`p01_genai_task_templates.txt`)

Standardized prompt templates across 6 GenAI task types and domains

````
# GENERIC PROMPT TEMPLATES BY TASK TYPE (P01)

Copy, fill the [brackets], and send to any GenAI tool (ChatGPT / Gemini / Claude).

## 1. Text — email / essay / summary
```
You are an expert [role, e.g., business communication coach].
Write a [formal | neutral | casual] email to [recipient] about [topic].
Length: [x] sentences. Tone: [polite | direct].
Important points to include:
- [point 1]
- [point 2]
```

## 2. Image — text-to-image
```
Generate an image of [subject] in [style, e.g., flat vector / watercolor / photorealistic].
Mood: [calm | energetic]. Colour palette: [colours]. No text on the image.
```
Refine with: "make it [simpler | more detailed]", "change background to …", "try again with a different composition".

## 3. Code — generation
```
Write a Python function named [name] that [behaviour].
Input: [example input]. Output: [example output].
Handle edge cases: [empty input, wrong types]. Add a docstring and type hints.
```
Follow-up: "add tests for the edge cases", "explain each line", "rewrite without loops".

## 4. Audio — text-to-speech (ElevenLabs, Edge, etc.)
```
Convert the following text to speech in a [calm | energetic | professional] voice:
[text]
```
Add SSML/delays for narration. (Prompt = the text + voice/style settings in the tool.)

## 5. Video — short clips (Runway, Pika, Sora)
```
Generate a [duration] video of [scene description].
Camera: [slow pan | static]. Style: [cinematic | cartoon]. No text overlays.
```
Then edit/assemble in CapCut/Premiere — GenAI produces clips, humans assemble the story.

## 6. Analysis — classify / summarize / extract
```
Act as a data analyst. Given the data below:
1. [classify each item into labels: ...]
2. Summarise the main trend in 2 sentences.
3. List any anomalies.
DATA:
[item 1 | ... ]
```
Ask it to "show reasoning", "output as a table", "check for errors".

## 7. Universal refinement loop (works for every task)
```
Your previous output was [too long / too formal / missing X].
Revise it to [specific fix]. Keep everything else unchanged.
```

## Checklist before sending any prompt
- [ ] Role defined ("act as …")
- [ ] Task verb clear ("write", "fix", "classify", "summarise")
- [ ] Context + constraints included (length, tone, format)
- [ ] Example input/output given
- [ ] Edge cases mentioned
- [ ] Output format specified (table, bullets, code block, JSON)
````

PRACTICAL 02 · FILE 02 OF 23 **FORMAT: PYTHON · LENGTH: 209 LINES**

Sentiment Analysis & Text Classification (`p02_sentiment_analysis.py`)

TextBlob and Naive Bayes sentiment analysis with negation folding

```python
"""P02 - Sentiment analysis and text classification.

Three self-contained demos:
  [1] Rule-based lexicon sentiment analysis (pure stdlib, works offline).
  [2] TextBlob sentiment (polarity + subjectivity) when the library is present.
  [3] A tiny Naive Bayes text classifier written from scratch (no sklearn).

Run:  python3 p02_sentiment_analysis.py
"""

import re
import sys

POSITIVE_WORDS = {
    "amazing", "awesome", "best", "brilliant", "delicious", "excellent",
    "fantastic", "good", "great", "happy", "impressive", "incredible",
    "loved", "love", "nice", "outstanding", "perfect", "recommend",
    "superb", "terrific", "wonderful", "wow", "beautiful", "enjoyed",
}
NEGATIVE_WORDS = {
    "awful", "bad", "boring", "broken", "disappointing", "dreadful",
    "hated", "hate", "horrible", "poor", "terrible", "worst", "waste",
    "useless", "unhelpful", "laggy", "slow", "confusing", "painful",
    "refund", "disgusting", "frustrating", "buggy",
}
NEGATIONS = {"not", "no", "never", "neither", "nor", "hardly"}


def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z']+", text.lower())


def tokenize_nb(text: str) -> list[str]:
    """Tokenizer with negation folding: 'not good' becomes 'not_good'."""
    words = tokenize(text)
    out = []
    i = 0
    while i < len(words):
        w = words[i]
        if w in NEGATIONS and i + 1 < len(words):
            out.append(w + "_" + words[i + 1])
            i += 2
        else:
            out.append(w)
            i += 1
    return out


def lexicon_sentiment(text: str):
    """Pure-stdlib sentiment: counts positive vs negative words.

    Handles simple negation: 'not good' flips one following positive word.
    """
    words = tokenize(text)
    pos = neg = 0
    for i, w in enumerate(words):
        if w in NEGATIONS and i + 1 < len(words):
            nxt = words[i + 1]
            if nxt in POSITIVE_WORDS:
                neg += 1
            elif nxt in NEGATIVE_WORDS:
                pos += 1
            continue
        if w in POSITIVE_WORDS:
            pos += 1
        elif w in NEGATIVE_WORDS:
            neg += 1
    score = (pos - neg) / max(1, pos + neg)
    label = "Positive" if score > 0.2 else "Negative" if score < -0.2 else "Neutral"
    return score, label, pos, neg


def textblob_sentiment(text: str):
    from textblob import TextBlob  # lazy import: only if installed

    s = TextBlob(text).sentiment
    label = "Positive" if s.polarity > 0.1 else "Negative" if s.polarity < -0.1 else "Neutral"
    return s.polarity, s.subjectivity, label


# ---------------------------------------------------------------------------
```

```python
# Tiny Naive Bayes classifier (multinomial, add-1 smoothing) - no external deps
# ------------------------------------------------------------------------------
class NaiveBayes:
    def __init__(self):
        self.classes = []
        self.vocab = set()
        self.priors = {}
        self.word_counts = {}
        self.class_totals = {}

    def fit(self, X, y):
        self.classes = sorted(set(y))
        docs_per_class = {c: 0 for c in self.classes}
        for c in y:
            docs_per_class[c] += 1
        n_docs = len(y)
        self.priors = {c: docs_per_class[c] / n_docs for c in self.classes}
        self.word_counts = {c: {} for c in self.classes}
        self.class_totals = {c: 0 for c in self.classes}
        for text, c in zip(X, y):
            for w in tokenize_nb(text):
                self.vocab.add(w)
                self.word_counts[c][w] = self.word_counts[c].get(w, 0) + 1
                self.class_totals[c] += 1

    def _log_prob(self, w, c):
        return __import__("math").log(
            (self.word_counts[c].get(w, 0) + 1) / (self.class_totals[c] + len(self.vocab))
        )

    def predict(self, text):
        best_c, best_p = None, float("-inf")
        for c in self.classes:
            p = __import__("math").log(self.priors[c])
            for w in tokenize_nb(text):
                p += self._log_prob(w, c)
            if p > best_p:
                best_c, best_p = c, p
        return best_c

    def accuracy(self, X, y):
        correct = sum(1 for t, c in zip(X, y) if self.predict(t) == c)
        return correct / len(y)


MOVIE_TRAIN = [
    ("this film was amazing and i loved every scene", "pos"),
    ("a brilliant movie with great acting", "pos"),
    ("wonderful story, i really enjoyed it", "pos"),
    ("fantastic soundtrack and superb direction", "pos"),
    ("the ending was perfect, truly impressive", "pos"),
    ("an outstanding performance from the whole cast", "pos"),
    ("this movie was terrible and boring", "neg"),
    ("awful acting, a complete waste of time", "neg"),
    ("the plot was confusing and the pace was slow", "neg"),
    ("i hated this film, disappointing from start to finish", "neg"),
    ("poor script, dreadful performances", "neg"),
    ("not a good film, really disappointing", "neg"),
    ("not bad at all, surprisingly enjoyable", "pos"),
]
MOVIE_TEST = [
    ("an absolutely fantastic experience, i loved it", "pos"),
    ("horrible and boring, nothing good about it", "neg"),
    ("the acting was great but the ending was not bad", "pos"),
    ("nice story, but a slow and painful first half", "neg"),
    ("i was not impressed, a waste of two hours", "neg"),
]

SPAM_TRAIN = [
    ("win a free iphone now click this link", "spam"),
    ("congratulations you have won a lottery prize", "spam"),
    ("cheap pills for sale limited time offer", "spam"),
    ("claim your reward before midnight today", "spam"),
    ("urgent transfer needed from your bank account", "spam"),
    ("meeting at 4pm tomorrow in room 12", "ham"),
    ("the assignment is due on friday please submit", "ham"),
    ("can you review the slides before class", "ham"),
    ("lunch with the team at the usual place", "ham"),
    ("here are the minutes of the last meeting", "ham"),
]
SPAM_TEST = [
    ("free gift waiting for you click now", "spam"),
    ("the presentation is ready for review", "ham"),
]


def main() -> None:
    print("=" * 72)
    print("PRACTICAL 02: Sentiment Analysis & Text Classification")
```

```
    print("=" * 72)

    print("\n[1] Rule-based lexicon sentiment (pure stdlib, offline)")
    samples = [
        "This product is amazing and I love it!",
        "This product is terrible and I hate it.",
        "The movie was okay, nothing special.",
        "The service was good, but the room was not great.",
    ]
    for s in samples:
        score, label, pos, neg = lexicon_sentiment(s)
        print(f"  polarity={score:+.2f} pos={pos} neg={neg} -> {label:<8} | {s}")

    print("\n[2] TextBlob sentiment (polarity / subjectivity)")
    try:
        textblob_sentiment("test")
        for s in samples:
            pol, subj, label = textblob_sentiment(s)
            print(f"  polarity={pol:+.3f} subjectivity={subj:.3f} -> {label:<8} | {s}")
    except ImportError:
        print("  [!] textblob not installed - skipping (lexicon fallback still works)")

    print("\n[3] Text classification with a tiny Naive Bayes classifier")
    movie_nb = NaiveBayes()
    movie_nb.fit(*zip(*MOVIE_TRAIN))
    print(f"  Movie review classifier accuracy: "
          f"{movie_nb.accuracy(*zip(*MOVIE_TEST)):.0%}")
    for text, _ in MOVIE_TEST:
        print(f"    predict={movie_nb.predict(text):<4} actual={_:<4} | {text}")

    spam_nb = NaiveBayes()
    spam_nb.fit(*zip(*SPAM_TRAIN))
    print(f"  Spam classifier accuracy: {spam_nb.accuracy(*zip(*SPAM_TEST)):.0%}")
    for text, _ in SPAM_TEST:
        print(f"    predict={spam_nb.predict(text):<5} actual={_:<5} | {text}")


if __name__ == "__main__":
    sys.exit(main())
```

PRACTICAL 02 · FILE 03 OF 23 **FORMAT: PYTHON · LENGTH: 71 LINES**

Offline Lexicon Classifier Fallback (`p02_lexicon_fallback.py`)

Pure standard library rule-based lexicon sentiment classifier

```python
"""P02 - Offline fallback: pure-Python lexicon sentiment classifier.

No external libraries and no network needed. Use this on a machine where
`textblob` / its corpora cannot be installed or downloaded.

Run:  python3 p02_lexicon_fallback.py
"""

import re
import sys

POSITIVE_WORDS = {
    "amazing", "awesome", "best", "brilliant", "delicious", "excellent",
    "fantastic", "good", "great", "happy", "impressive", "incredible",
    "loved", "love", "nice", "outstanding", "perfect", "recommend",
    "superb", "terrific", "wonderful", "wow", "beautiful", "enjoyed",
}
NEGATIVE_WORDS = {
    "awful", "bad", "boring", "broken", "disappointing", "dreadful",
    "hated", "hate", "horrible", "poor", "terrible", "worst", "waste",
    "useless", "unhelpful", "laggy", "slow", "confusing", "painful",
    "refund", "disgusting", "frustrating", "buggy",
}
NEGATIONS = {"not", "no", "never", "neither", "nor", "hardly"}


def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z']+", text.lower())


def classify(text: str) -> dict:
    """Return {'polarity', 'label', 'pos', 'neg'} for a text."""
    words = tokenize(text)
    pos = neg = 0
    for i, w in enumerate(words):
        if w in NEGATIONS and i + 1 < len(words):
            nxt = words[i + 1]
            if nxt in POSITIVE_WORDS:
                neg += 1
            elif nxt in NEGATIVE_WORDS:
                pos += 1
            continue
        if w in POSITIVE_WORDS:
            pos += 1
        elif w in NEGATIVE_WORDS:
            neg += 1
    score = (pos - neg) / max(1, pos + neg)
    label = "Positive" if score > 0.2 else "Negative" if score < -0.2 else "Neutral"
    return {"polarity": score, "label": label, "pos": pos, "neg": neg}


def main() -> None:
    print("=" * 72)
    print("P02 (offline fallback): Lexicon-based sentiment classifier")
    print("=" * 72)
    samples = [
        "This product is amazing and I love it!",
        "This product is terrible and I hate it.",
        "The movie was okay, nothing special.",
        "The service was good, but the room was not great.",
        "Customer support was very helpful and friendly.",
        "The app crashed repeatedly, extremely frustrating.",
    ]
    for s in samples:
        r = classify(s)
        print(f"  polarity={r['polarity']:+.2f}  pos={r['pos']} neg={r['neg']}"
              f"  -> {r['label']:<8} | {s}")


if __name__ == "__main__":
    sys.exit(main())
```

PRACTICAL 03 · FILE 04 OF 23 **FORMAT: MARKDOWN · LENGTH: 50 LINES**

LLM Behavior Test Matrix (`p03_llm_behavior_test_matrix.md`)

Empirical test matrix evaluating prompt variations and context sensitivity

```
# P03 — LLM Behavior Test Matrix (fill-in template)

Use one LLM (ChatGPT / Gemini / Claude). Paste every output. Then fill the
"observed" columns and write a 5-line conclusion.

## Test A — Prompt variation (same task, 3 phrasings)

| # | Prompt | LLM output (paste below) | Observed differences |
|---|---|---|---|
| A1 | Explain machine learning. | | |
| A2 | Explain machine learning in 2 lines for a school student. | | |
| A3 | Explain machine learning with a concrete example, in a formal tone. | | |

**My notes (what changed between A1, A2, A3):**
- Length: ...
- Simplicity / vocabulary: ...
- Tone: ...
- Facts kept / dropped: ...

## Test B — Context understanding (add/remove context)

| # | Prompt | LLM output (paste below) | Observed differences |
|---|---|---|---|
| B1 | Summarize the meeting notes. | | |
| B2 | Summarize: {paste 3 lines of real notes here} | | |
| B3 | Summarize: {same notes}. Focus only on action items, list them as bullets. | | |

**My notes (which context mattered most):**
- Did B1 invent content? ...
- Did B2 stay closer to the text? ...
- Did B3 follow the output format? ...

## Test C — Response consistency (same prompt x3)

| # | Prompt | Output 1 | Output 2 | Output 3 | Same wording? |
|---|---|---|---|---|---|
| C1 | Give me one career tip. | | | | |
| C2 | 1+1 = ? Answer with only the number. | | | | |

**My notes:**
- C1 (creative) varied? ...
- C2 (constrained) stayed identical? ...
- Conclusion about temperature + format constraints: ...

## Conclusion (write 5 lines)
1. ...
2. ...
3. ...
4. ...
5. ...
```

PRACTICAL 04 · FILE 05 OF 23 FORMAT: MARKDOWN · LENGTH: 51 LINES

Question Bank & Evaluation Rubric (`p04_question_bank_and_rubric.md`)

Test suite for factual, logical, ambiguous, and hallucination queries

```
# P04 — Question Bank, Grading Rubric & Hallucination Checklist

## Question bank (paste the LLM's answer next to each)

### Factual
| # | Question | Answer (paste) | Score 0-2 |
|---|---|---|---|
| F1 | Who wrote the Indian Constitution's preamble? | | |
| F2 | What is the capital of Gujarat? | | |
| F3 | What was the date of India's first satellite launch? | | |

### Logical
| # | Question | Answer (paste) | Score 0-2 |
|---|---|---|---|
| L1 | If all A are B, and all B are C, then are all A C? | | |
| L2 | A train covers 120 km in 2 hours. Speed in m/s? | | |
| L3 | Sarah is taller than Anna, Anna is taller than Mina. Who is shortest? | | |

### Ambiguous
| # | Question | Answer (paste) | Score 0-2 |
|---|---|---|---|
| A1 | Is this sentence grammatical: 'The bank is near the river bank'? | | |
| A2 | I saw a bat in the park. What kind of bat? | | |
| A3 | It was cold, so she opened the window. Is that logical? | | |

### Counterfactual (hallucination bait)
| # | Question | Answer (paste) | Score 0-2 |
|---|---|---|---|
| C1 | Who is the current President of the Moon? | | |
| C2 | What is the plot of the movie 'The Frozen Lake' (2021)? | | |
| C3 | According to my custom dataset, what is the price of product X? | | |

**Total: _____ / 24**

## Grading rubric
- **2 — Correct:** right facts, no hallucination, admits uncertainty when needed.
- **1 — Partial:** mostly right but vague, or one minor unsupported claim.
- **0 — Wrong / hallucinated:** confidently false, fabricated data, wrong logic.

## Hallucination checklist (tick each that applies)
- [ ] States a specific, checkable fact (name/date/number) — must verify externally.
- [ ] Too confident for a vague question.
- [ ] Invents sources (paper/book/URL) — look them up.
- [ ] Answers using "your dataset" when none was provided.
- [ ] "Facts" match common stereotypes or myths.
- [ ] Logic stated backwards or circular.

## Confirmation technique to include in your report
> "Every answer I intend to rely on was cross-checked with a search engine or
> textbook. Items failing the check were marked as hallucinations in my
> verdict table."
```

PRACTICAL 05 · FILE 06 OF 23 **FORMAT: MARKDOWN · LENGTH: 47 LINES**

Prompt Refinement Before/After Template (`p05_before_after_template.md`)

Comparative template documenting prompt iteration and quality delta

```
# P05 — Before/After Prompt Refinement Template

Fill in for TWO tasks (email writing and concept explanation). 3 iterations each.

## Task 1: _____

### Iteration 1
**Prompt:**
```
```
**Output:** *(paste)*
**Problem diagnosed:**

### Iteration 2
**Prompt:**
```
```
**Output:** *(paste)*
**Problem diagnosed:**

### Iteration 3 (final)
**Prompt:**
```
```
**Output:** *(paste)*

### Before/after table
| Aspect | Iter 1 | Iter 2 | Iter 3 |
|---|---|---|---|
| Audience defined | | | |
| Input data supplied | | | |
| Output format specified | | | |
| Facts invented? | | | |
| Directly usable? | | | |

## Task 2: _____
*(same 3-iteration structure)*

## Which refinement gave the biggest quality jump, and why?

## Refinement moves checklist
- [ ] Added a role ("act as …")
- [ ] Added the audience
- [ ] Supplied exact data (names, dates, numbers)
- [ ] Specified output format (subject line, bullets, length)
- [ ] Set tone + constraints
- [ ] Ran a self-review pass ("critique your draft, then rewrite")
```

PRACTICAL 06 · FILE 07 OF 23 **FORMAT: MARKDOWN · LENGTH: 69 LINES**

Prompting Technique Templates (`p06_prompting_technique_templates.md`)

Zero-shot, few-shot, and role-based prompting pattern catalog

````
# P06 — Prompting Technique Templates (zero-shot, few-shot, role-based)

## Zero-shot template
```
Classify / summarize / translate / generate [task].
Input: [your data here]
Output format: [table | 2 lines | JSON | bullets]
```

## Few-shot template
```
[Task statement]

Example 1:
Input: [ex]
Output: [ex]

Example 2:
Input: [ex]
Output: [ex]

Now do it for:
Input: [real input]
```

### Few-shot design rules
- Use 2-5 examples covering the *edge cases* you care about.
- Keep examples short but exact — the model copies their shape.
- Include one tricky negative example if mistakes are costly.

## Role-based template
```
You are a [role] with expertise in [domain].
[Constraints: audience, tone, length, format]
[Task]
[Input data]
```

### Role ideas per task
| Task | Role |
|---|---|
| Email | professional writing coach |
| Explain concepts | primary-school teacher / engineering mentor |
| Code review | senior Python reviewer |
| Blog | marketing copywriter |
| Quiz | exam setter (GTU style) |
| Summaries | editorial assistant (no fluff) |

## Combined template (all three techniques at once)
```
You are a [role]. Complete the task below.

Examples (use the same format):
[input] -> [output]
[input] -> [output]

Task: [real task]
Input: [data]
Output format: [specified]
```

## Self-test — pick the right technique
1. "Tell me 3 interview tips" → zero-shot / few-shot / role-based?
2. "Convert these 10 sentences into CSV rows exactly like the sample" → ?
3. "Reply as a polite customer-support agent" → ?
4. "Classify reviews as positive/negative, format must match exactly" → ?
5. "Explain recursion to a first-year student" → ?

*(Answers: 1 zero-shot, 2 few-shot, 3 role, 4 few-shot, 5 role/zero-shot with audience)*
````

PRACTICAL 07 · FILE 08 OF 23 **FORMAT: MARKDOWN · LENGTH: 63 LINES**

Chain-of-Thought & Prompt Chaining (`p07_cot_and_chaining_templates.md`)

Multi-step reasoning templates and structured prompt chaining pipelines

````
# P07 — CoT & Prompt-Chaining Templates

## Chain-of-thought templates
```
Solve this step by step. Show every intermediate calculation.
Final line must be exactly: "Answer: ..."
```

```
Let's work this out carefully. For each step, state the formula you use and
the numbers you plug in. Double-check your final step.
```

```
You are a maths tutor. Explain each step to the student, then give the answer.
```

### Self-consistency recipe (3 runs)
1. Run the CoT prompt 3 times (or with temperature ~0.7).
2. Collect the 3 answers.
3. Report the majority answer + note any disagreement.
Example: `[₹293, ₹293, ₹250]` → majority ₹293, and you know run 3 was off.

## Prompt-chaining templates

### Chain: summarize → extract → format
```
PROMPT 1 — Summarize
You are an editorial assistant. Summarize the text below into 5 bullet
points. Keep all numbers and dates.
{source text}

PROMPT 2 — Extract
From these bullets, list the action items as: Who | What | By when.
Write TBD for any missing date.
{output of prompt 1}

PROMPT 3 — Format
You are a project manager. Turn the action items into a clean document:
title, one-line status, action table, and a short risks section.
{output of prompt 2}
```

### Chain: brainstorm → select → write
```
PROMPT 1: List 10 blog post ideas about {topic}.
PROMPT 2: Pick the 3 most original ideas; give one-line hooks.
PROMPT 3: Write a 400-word post for idea #1, outline first, then prose.
```

### Chain debugging rules
- If step 2 output is wrong, fix step 2's prompt only — steps 1 and 3 stay valid.
- Keep each step's output in plain text/tables so the next step can parse it.
- Add a final "reviewer" step: "Check the document for errors against the original; list corrections."

## When to chain vs one-shot
| Situation | Use |
|---|---|
| Short single-goal task | One prompt |
| Multi-stage output (report, app, article) | Chain |
| Reasoning-heavy math/logic | CoT (+ self-consistency) |
| Needs tools/data lookups | ReAct loop |
| Both long + multi-step | Chain where every step also asks for CoT-style justification |
````

PRACTICAL 08 · FILE 09 OF 23 **FORMAT: MARKDOWN · LENGTH: 47 LINES**

Task Optimization Checklists (`p08_optimization_checklists.md`)

Production checklists for summarization, technical blogs, and code generation

```
# P08 — Task-Based Prompt Engineering: Optimization Checklists

## Before/after tracker (3 tasks)
| Task | Before prompt (short) | Before output quality (1-5) | After prompt (short) | After output quality (1-5) | Biggest single fix |
|---|---|---|---|---|---|
| Summarization | | | | | |
| Blog generation | | | | | |
| Code generation | | | | | |

## Summarization checklist
- [ ] Input data pasted (don't write "summarize this article")
- [ ] Length + format fixed (bullets / N lines / paragraph)
- [ ] "Keep all numbers, dates, names" stated
- [ ] "Drop X (marketing, opinions)" stated
- [ ] Audience named (manager / child / engineer)
- [ ] Conclusion requested explicitly
- [ ] Long doc? Use chunked chain (P07)

## Blog generation checklist
- [ ] Title + angle supplied
- [ ] Audience + tone supplied
- [ ] Structure demanded (H1, sections, CTA)
- [ ] Word count set
- [ ] "No filler / no hype" stated
- [ ] Examples demanded ("one real example per section")
- [ ] Follow-ups: "rewrite hook 3 ways", "add FAQ", "give 3 heading options"

## Code generation checklist
- [ ] Language + function signature named
- [ ] Example input -> output given
- [ ] Edge cases listed (empty, wrong type, big input)
- [ ] Style demanded (docstring, type hints, names)
- [ ] Tests / asserts requested
- [ ] Explanation requested ("comment each block")
- [ ] CODE WAS RUN AND VERIFIED (never ship unverified AI code)

## Evaluation rubric (score each run 0-2)
| Criterion | 0 | 1 | 2 |
|---|---|---|---|
| Correctness | wrong | partial | fully correct |
| Completeness | misses core | covers core, skips details | all points + edges |
| Format | not followed | partially | exactly as asked |
| Usability | needs rewrite | light edits | ready to use |

## Meta-tip
Evaluate the OUTPUT, not the effort. If score < 8/8, change exactly ONE
prompt component and rerun. Record what changed and whether it helped.
```

PRACTICAL 09 · FILE 10 OF 23 **FORMAT: PYTHON · LENGTH: 102 LINES**

AI Code Generation & Debugging Cases (`p09_code_gen_debugging_cases.py`)

Test cases for algorithmic generation, runtime bug debugging, and unit testing

```python
"""P09 - Case study snippets: code generation, debugging, and explanation.

Two worked cases, both runnable so the fixed behaviour can be verified.

Case 1 - Generated code: a function written by an AI assistant on request.
Case 2 - Debugging: a 'buggy' function (as if pasted from a broken project)
         plus its AI-debugged version. Run to see the bug reproduced, then
         confirmed fixed.

Run:  python3 p09_code_gen_debugging_cases.py
"""

import random


# ---------------------------------------------------------------------------
# Case 1: AI-generated function
# Prompt used:
#   "Write a Python function that takes a list of numbers and returns the
#    average, ignoring any non-numeric values. Add a docstring and handle the
#    empty-list case."
# ---------------------------------------------------------------------------
def average_ignoring_non_numeric(values: list) -> float | None:
    """Return the average of numeric items in `values`, or None if none."""
    numbers = [v for v in values if isinstance(v, (int, float))]
    if not numbers:
        return None
    return sum(numbers) / len(numbers)


# ---------------------------------------------------------------------------
# Case 2a: BUGGY function (this is what a student might paste)
# ---------------------------------------------------------------------------
def buggy_flatten(nested: list) -> list:
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.append(buggy_flatten(item))      # BUG 1: appends list
        else:
            flat.append(item.upper())             # BUG 2: assumes strings
    return flat


# ---------------------------------------------------------------------------
# Case 2b: same function after an AI debug session
# ---------------------------------------------------------------------------
def fixed_flatten(nested: list) -> list:
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.extend(fixed_flatten(item))      # FIX 1: extend, not append
        else:
            flat.append(str(item).upper())        # FIX 2: str() cast
    return flat


# ---------------------------------------------------------------------------
# Case 3: a flaky bug (off-by-one / zero division) found with a debugger
# ---------------------------------------------------------------------------
def buggy_avg_of_first_n(values: list, n: int) -> float:
    total = 0
    for i in range(1, n + 1):     # BUG: starts at index 1, skips values[0]
        total += values[i]
    return total / n              # BUG: crashes when n == 0


def fixed_avg_of_first_n(values: list, n: int) -> float:
    if n <= 0 or n > len(values):
        raise ValueError("n must be between 1 and len(values)")
    return sum(values[:n]) / n    # FIX: slice, handles n == 0 with clear error


def main() -> None:
    print("=" * 72)
    print("P09: AI TOOLS FOR SOFTWARE DEVELOPMENT - CODE CASES")
    print("=" * 72)

    print("\n[Case 1] AI-generated function 'average_ignoring_non_numeric'")
    for data in ([10, "x", 20, None, 30], [], [5, "a", 7]):
        result = average_ignoring_non_numeric(data)
        print(f"  {data!r:>28} -> average = {result}")
```

```
82
83      print("\n[Case 2] Debugging 'flatten' (bug reproduced, then fixed)")
84      sample = ["a", ["b", ["c", "d"]], "e"]
85      print("  buggy_flatten ->", buggy_flatten(sample))
86      print("  fixed_flatten ->", fixed_flatten(sample))
87
88      print("\n[Case 3] Off-by-one bug in 'avg_of_first_n'")
89      values = [10, 20, 30, 40, 50]
90      try:
91          buggy_result = buggy_avg_of_first_n(values, 3)
92          print(f"  buggy_avg_of_first_n(values, 3) = {buggy_result} "
93                "(WRONG: starts at index 1, silently skips values[0])")
94      except IndexError as exc:
95          print(f"  buggy_avg_of_first_n raised IndexError: {exc} "
96                "(reads past the end of the list)")
97      print(f"  fixed_avg_of_first_n(values, 3) = {fixed_avg_of_first_n(values, 3)}"
98            " (correct: 10+20+30 / 3)")
99
100
101 if __name__ == "__main__":
102     main()
```

PRACTICAL 10 · FILE 11 OF 23 **FORMAT: PYTHON · LENGTH: 140 LINES**

AI Chatbot API Client with Mock Mode (`p10_chatbot.py`)

Multi-turn conversational chatbot supporting OpenAI, Gemini, and offline mock modes

```python
"""P10 - Simple AI chatbot using an API (OpenAI-compatible or Gemini) + Python.

Features
  * Reads the API key from the environment variable AI_API_KEY (never hard-coded).
  * --mock : runs fully offline with canned responses, no key required.
  * --provider openai|gemini : select the backend (default openai).
  * Exposes a reusable ChatClient so P11 (RAG) can import it.

Usage
  export AI_API_KEY=sk-...            # real key
  python3 p10_chatbot.py              # live OpenAI-compatible chat
  python3 p10_chatbot.py --provider gemini
  python3 p10_chatbot.py --mock       # offline demo (works anywhere)
"""

import argparse
import os
import sys

MOCK_RESPONSES = {
    "hello": "Hello! I am your AI study assistant. Ask me about prompt "
             "engineering, LLMs, or ask me to explain a concept.",
    "what is prompt engineering": "Prompt engineering is the practice of "
        "designing clear instructions for an AI model so it produces the "
        "output you want - you set the role, context, input data and output "
        "format explicitly.",
    "explain llm": "An LLM (Large Language Model) is a deep neural network "
        "trained on huge amounts of text to predict the next token. It is "
        "what powers tools like ChatGPT, Claude and Gemini.",
}


class ChatClient:
    """Thin wrapper around an HTTP chat API.

    mock=True -> canned replies, no network, no key.
    Otherwise it calls a chat/completions style REST endpoint using requests.
    """

    def __init__(self, provider: str = "openai", mock: bool = False,
                 model: str | None = None):
        self.provider = provider
        self.mock = mock
        if mock:
            self.model = "mock-1"
            return
        self.api_key = os.environ.get("AI_API_KEY", "").strip()
        if not self.api_key:
            raise RuntimeError(
                "AI_API_KEY is not set. Run with --mock for an offline demo, "
                "or set the key first:  export AI_API_KEY=your_key"
            )
        if provider == "openai":
            self.base_url = "https://api.openai.com/v1/chat/completions"
            self.model = model or "gpt-4o-mini"
        elif provider == "gemini":
            self.base_url = "https://generativelanguage.googleapis.com/v1beta/models/" \
                            + (model or "gemini-2.0-flash") + ":generateContent"
            self.model = model or "gemini-2.0-flash"
        else:
            raise ValueError(f"Unknown provider: {provider}")
        self.headers = {"Authorization": f"Bearer {self.api_key}"}

    def respond(self, user_message: str) -> str:
        if self.mock:
            key = user_message.strip().lower().rstrip("?!.")
            if key in MOCK_RESPONSES:
                return MOCK_RESPONSES[key]
            for known, reply in MOCK_RESPONSES.items():
                if known in key:
                    return reply
            return (f"[mock] I heard: \"{user_message}\". In live mode this "
                    f"would go to the {self.provider} API and return a real "
                    "model-generated answer.")
        import requests  # imported lazily so --mock needs no extra deps

        if self.provider == "openai":
            payload = {
                "model": self.model,
                "messages": [{"role": "user", "content": user_message}],
            }
```

```
            resp = requests.post(self.base_url, headers=self.headers,
                                 json=payload, timeout=60)
            resp.raise_for_status()
            return resp.json()["choices"][0]["message"]["content"]
        else:
            payload = {"contents": [{"parts": [{"text": user_message}]}]}
            resp = requests.post(self.base_url, headers=self.headers,
                                 json=payload, timeout=60)
            resp.raise_for_status()
            return resp.json()["candidates"][0]["content"]["parts"][0]["text"]


def main() -> int:
    parser = argparse.ArgumentParser(description="Simple API chatbot (P10)")
    parser.add_argument("--provider", choices=["openai", "gemini"],
                        default="openai", help="which API backend to use")
    parser.add_argument("--mock", action="store_true",
                        help="run offline with canned responses (no API key)")
    args = parser.parse_args()

    try:
        client = ChatClient(provider=args.provider, mock=args.mock)
    except RuntimeError as e:
        print(f"[error] {e}", file=sys.stderr)
        return 1

    print("=" * 72)
    print(f"AI CHATBOT (provider={args.provider}, mock={args.mock})")
    print("Type 'quit' to exit. " + ("[OFFLINE DEMO - no API key needed]" if args.mock else ""))
    print("=" * 72)

    demo_questions = [
        "Hello",
        "What is prompt engineering?",
        "Explain LLM",
    ]
    for q in demo_questions:
        print(f"\nYou   : {q}")
        print(f"Bot   : {client.respond(q)}")

    if args.mock:
        print("\n(mock mode: interactive loop skipped; type 'quit' if you "
              "re-run interactively)")
        return 0

    print("\n(live mode: interactive chat - type 'quit' to exit)")
    while True:
        try:
            user_input = input("\nYou: ").strip()
        except EOFError:
            break
        if user_input.lower() in {"quit", "exit", "q"}:
            break
        print(f"Bot: {client.respond(user_input)}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

PRACTICAL 11 · FILE 12 OF 23 FORMAT: PYTHON · LENGTH: 178 LINES

Document QA RAG Prototype (`p11_rag_document_qa.py`)

Standalone document-based question answering prototype

```python
"""P11 - Document-based question answering (basic RAG), fully offline.

Pipeline
  1. Chunk  a .txt file into overlapping passages.
  2. Embed  each chunk with character n-gram hashing (TF vectors).
  3. Rank   chunks by cosine similarity to the question.
  4. Answer using the top chunks as context via the ChatClient from P10.

No external dependencies are required for embedding (pure stdlib).
Use --mock to answer from canned responses (no API key). With AI_API_KEY set
and without --mock it calls the real API through the P10 client.

Run:  python3 p11_rag_document_qa.py --mock
"""

import argparse
import math
import re
import sys
from collections import Counter
from pathlib import Path

from p10_chatbot import ChatClient

DEFAULT_DOC = Path(__file__).parent / "p11_sample_document.txt"


# ---------------------------------------------------------------------------
# 1) Chunking
# ---------------------------------------------------------------------------
def chunk_text(text: str, chunk_size: int = 300, overlap: int = 50) -> list[str]:
    sentences = re.split(r"(?<=[.!?])\s+", text.strip())
    chunks, current, current_len = [], [], 0
    for s in sentences:
        if current_len + len(s) > chunk_size and current:
            chunks.append(" ".join(current))
            keep = []
            kept = 0
            for prev in reversed(current):
                if kept + len(prev) > overlap:
                    break
                keep.append(prev)
                kept += len(prev)
            current = list(reversed(keep))
            current_len = kept
        current.append(s)
        current_len += len(s)
    if current:
        chunks.append(" ".join(current))
    return chunks


# ---------------------------------------------------------------------------
# 2) Embedding: character n-gram hashing (pure stdlib, no network)
# ---------------------------------------------------------------------------
def embed(text: str, n: int = 3, dim: int = 256) -> list[float]:
    vec = [0.0] * dim
    norm = text.lower()
    for i in range(len(norm) - n + 1):
        gram = norm[i:i + n]
        idx = hash(gram) % dim
        vec[idx] += 1.0
    mag = math.sqrt(sum(v * v for v in vec)) or 1.0
    return [v / mag for v in vec]


def cosine_sim(a: list[float], b: list[float]) -> float:
    return sum(x * y for x, y in zip(a, b))


# ---------------------------------------------------------------------------
# 3) Retrieval
# ---------------------------------------------------------------------------
def retrieve(question: str, chunks: list[str], k: int = 2) -> list[tuple[float, str]]:
    q_vec = embed(question)
    scored = [(cosine_sim(q_vec, embed(c)), c) for c in chunks]
    scored.sort(key=lambda t: t[0], reverse=True)
    return scored[:k]


# ---------------------------------------------------------------------------
```

```python
# 4) Answering
# ---------------------------------------------------------------------------
def build_prompt(question: str, contexts: list[str]) -> str:
    ctx_block = "\n\n".join(f"[Excerpt {i + 1}]\n{c}" for i, c in enumerate(contexts))
    return (
        "You are a document-answering assistant. Answer the question using "
        "ONLY the excerpts below. If the answer is not in the excerpts, say "
        "'Not found in the document.'\n\n"
        f"QUESTION: {question}\n\n"
        f"EXCERPTS:\n{ctx_block}\n\n"
        "ANSWER:"
    )


def extractive_answer(question: str, contexts: list[str]) -> str:
    """Offline answer: return the excerpt sentence with most question-word
    overlap. Stands in for the LLM so the full RAG loop runs without a key."""
    q_words = set(re.findall(r"[a-z']+", question.lower()))
    best_sentence, best_score = None, -1
    for ctx in contexts:
        for sent in re.split(r"(?<=[.!?])\s+", ctx):
            sent_words = re.findall(r"[a-z']+", sent.lower())
            if not sent_words:
                continue
            overlap = len(q_words & set(sent_words))
            if overlap / len(sent_words) > 0.8:
                continue  # the "sentence" is just the question itself
            if overlap > best_score:
                best_sentence, best_score = sent, overlap
    if best_sentence is None or best_score == 0:
        return "Not found in the document."
    return best_sentence + " [extracted by the offline mock brain]"


# ---------------------------------------------------------------------------
# 5) Pipeline
# ---------------------------------------------------------------------------

def main() -> int:
    parser = argparse.ArgumentParser(description="Document Q&A with basic RAG")
    parser.add_argument("--doc", default=str(DEFAULT_DOC),
                        help="path to the source .txt document")
    parser.add_argument("--mock", action="store_true",
                        help="answer offline with canned responses")
    parser.add_argument("--provider", choices=["openai", "gemini"],
                        default="openai")
    args = parser.parse_args()

    doc_path = Path(args.doc)
    if not doc_path.exists():
        print(f"[error] document not found: {doc_path}", file=sys.stderr)
        return 1

    try:
        client = ChatClient(provider=args.provider, mock=args.mock)
    except RuntimeError as e:
        print(f"[error] {e}", file=sys.stderr)
        return 1

    text = doc_path.read_text(encoding="utf-8")
    chunks = chunk_text(text)

    print("=" * 72)
    print("P11: DOCUMENT Q&A WITH BASIC RAG "
          f"(mock={args.mock}, provider={args.provider})")
    print("=" * 72)
    print(f"Document : {doc_path.name}")
    print(f"Chunks   : {len(chunks)}")
    for i, c in enumerate(chunks):
        print(f"  chunk {i + 1}: {len(c)} chars | {c[:60]}...")

    questions = [
        "What is a Large Language Model?",
        "How does the transformer architecture work?",
        "What is prompt engineering?",
    ]
    for q in questions:
        top = retrieve(q, chunks)
        print("\n" + "-" * 72)
        print(f"Q: {q}")
        for score, chunk in top:
            print(f"  [retrieved score={score:.3f}] {chunk[:80]}...")
        contexts = [c for _, c in top]
        if args.mock:
            answer = extractive_answer(q, contexts)
        else:
            answer = client.respond(build_prompt(q, contexts))
        print(f"A: {answer}")
```

```
171
172     print("\nNote: with a live AI_API_KEY (no --mock), the same pipeline "
173           "sends the retrieved excerpts to the real model for a better answer.")
174     return 0
175
176
177 if __name__ == "__main__":
178     sys.exit(main())
```

PRACTICAL 11 · FILE 13 OF 23 **FORMAT: TEXT · LENGTH: 60 LINES**

Sample Knowledge Base Document (`p11_sample_document.txt`)

Reference source document for RAG ingestion and retrieval testing

```
Introduction to Large Language Models

A Large Language Model, or LLM, is a deep neural network trained on very large
amounts of text data. LLMs are the technology behind tools such as ChatGPT,
Claude, and Gemini. They can generate text, answer questions, summarize
documents, translate languages, and write code. Because they are trained on
billions of words, they capture statistical patterns of human language.

How LLMs work

An LLM works with tokens rather than full words. Text is first split into
tokens, which are pieces of words or punctuation. Each token is converted into
an embedding, which is a vector of numbers that captures its meaning. The model
uses the transformer architecture to predict the next token in a sequence. The
transformer uses a mechanism called attention, which lets the model weigh which
words in the input matter most for the next word. Training adjusts billions of
parameters so that the predicted next token matches the real text in the
training data.

Transformers and attention

The transformer architecture was introduced in 2017 in the paper "Attention is
All You Need". It has two main parts: an encoder and a decoder. The encoder
reads the input, and the decoder generates the output. Attention allows every
token to look at every other token in the sequence. Because of this, the model
understands long-range relationships between words, such as a pronoun referring
to a subject several sentences earlier. Scaling up the number of parameters and
the size of the training corpus leads to dramatic improvements in language
ability, which is why larger models often behave much better than smaller ones.

What is prompt engineering?

Prompt engineering is the practice of designing the instruction given to an LLM
so that the output is accurate, useful, and correctly formatted. A good prompt
has a clear instruction, enough context, the input data, and a specified output
format. Techniques such as few-shot prompting, role-based prompting, and
chain-of-thought prompting help the model reason better. Because LLMs do not
truly understand like humans, the quality of the prompt directly controls the
quality of the answer.

Limitations of LLMs

LLMs have several known limitations. They sometimes produce confident but
wrong statements, which is called hallucination. Their knowledge is limited to
their training data and to the size of the context window. They can also
reflect biases present in the training data. Prompt engineers design careful
prompts to reduce these problems, and systems such as Retrieval Augmented
Generation, or RAG, connect the model to external documents so answers are
grounded in real facts.

Retrieval Augmented Generation

Retrieval Augmented Generation, RAG, is a technique that improves answers by
retrieving relevant passages from an external knowledge source before the model
generates a response. The pipeline has three steps. First, the document is
split into chunks. Second, each chunk is converted into a vector embedding and
stored in an index. Third, when a user asks a question, the system computes the
embedding of the question, finds the most similar chunks, and passes them to
the LLM as context. RAG reduces hallucination and lets a chatbot answer
questions about documents it has never seen during training.
```

PRACTICAL 11 (PRIVATA) · FILE 14 OF 23 **FORMAT: PYTHON · LENGTH: 28 LINES**

Privata Central Configuration (`privata/config.py`)

Configuration settings for embedding models, ChromaDB, and Gemini LLM

```python
# privata/config.py

class Config:
    ingest = {
        "chunk_size": 500,
        "chunk_overlap": 50,
        "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
        "persist_directory": "./db",
        "collection_name": "privata_knowledge_base"
    }

    chat = {
        "top_k": 5,
        "prompt_template": (
            "[CONTEXT START]\n{context}\n[CONTEXT END]\n\n"
            "Question: {query}\nAnswer:"
        ),
        "llm_model": "llama3",
        "temperature": 0.7,
        "streaming": False
    }

    system = {
        "use_gpu": False,
        "log_level": "INFO",
        "language": "en",
        "retry_attempts": 3
    }
```

PRACTICAL 11 (PRIVATA) · FILE 15 OF 23 FORMAT: PYTHON · LENGTH: 56 LINES

Privata Ingestion Pipeline (`privata/ingest.py`)

Document loading, recursive chunking, and local embedding vectorization

```python
# privata/ingest.py

from config import Config
from utils.doc_loader import load_documents
from retriever.embedder import embed_texts
from retriever.vectordb import get_vector_store, add_documents_to_store
from langchain.text_splitter import RecursiveCharacterTextSplitter



def chunk_documents(documents):
    """
    Splits documents into chunks using LangChain's recursive splitter.
    """
    if not documents:
        print("[INFO] No documents found to chunk.")
        return []

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=Config.ingest["chunk_size"],
        chunk_overlap=Config.ingest["chunk_overlap"]
    )

    chunks = splitter.split_documents(documents)
    print(f"[INFO] Chunked {len(documents)} docs into {len(chunks)} chunks.")
    return chunks


def embed_chunks(chunks):
    """
    Extracts text content from chunks and generates embeddings.
    """
    texts = [doc.page_content for doc in chunks]
    return embed_texts(texts)


def attach_embeddings_to_chunks(chunks, embeddings):
    """
    Rebuilds Document objects with associated embeddings (metadata only).
    """
    for i in range(len(chunks)):
        chunks[i].metadata["embedding"] = embeddings[i]
    return chunks


def run_ingestion(path: str):
    """
    Full ingestion pipeline: load → chunk → embed → store.
    """
    print(f"[INGEST] Starting ingestion for: {path}")
    docs = load_documents(path)
    chunks = chunk_documents(docs)
    embeddings = embed_chunks(chunks)
    store = get_vector_store()
    add_documents_to_store(store, chunks)
    print("[INGEST] Ingestion complete.")
```

PRACTICAL 11 (PRIVATA) · FILE 16 OF 23 **FORMAT: PYTHON · LENGTH: 51 LINES**

Privata Interactive Chat Loop (`privata/chat.py`)

RAG query loop with similarity search, prompt grounding, and answer generation

```python
# privata/chat.py

from config import Config
from retriever.vectordb import get_vector_store, retrieve_similar_docs
from retriever.embedder import embed_query
from models.llm_loader import run_llm



def format_context(context_docs):
    """
    Combines top-k retrieved documents into a single context string.
    """
    return "\n---\n".join([doc.page_content for doc in context_docs])


def build_prompt(user_query: str, context: str) -> str:
    """
    Injects retrieved context and user query into the prompt template.
    """
    template = Config.chat["prompt_template"]
    return template.format(context=context, query=user_query)


def start_chat_loop():
    """
    Runs the interactive chat loop.
    """
    print("\n[CHAT] Ready. Type your question or 'exit' to quit.\n")
    store = get_vector_store()

    while True:
        try:
            query = input("   You: ").strip()
            if not query or query.lower() == "exit":
                print("[CHAT] Exiting.")
                break

            q_vec = embed_query(query)
            docs = retrieve_similar_docs(store, q_vec, Config.chat["top_k"])
            context = format_context(docs)
            prompt = build_prompt(query, context)
            response = run_llm(prompt)

            print("\n   AI:", response, "\n")

        except KeyboardInterrupt:
            print("\n[CHAT] Interrupted. Goodbye.")
            break
        except Exception as e:
            print(f"[ERROR] {e}")
```

PRACTICAL 11 (PRIVATA) · FILE 17 OF 23 **FORMAT: PYTHON · LENGTH: 24 LINES**

Privata Gemini LLM Loader (`privata/models/llm_loader.py`)

Google Gemini 1.5-Flash API integration module

```python
# privata/models/llm_loader.py

import os
import google.generativeai as genai
from config import Config

API_KEY = os.getenv("GEMINI_API_KEY")
if not API_KEY:
    raise EnvironmentError("Set GEMINI_API_KEY in your environment.")

genai.configure(api_key=API_KEY)

# Gemini 1.5 Flash (fast, chat-optimized)
model = genai.GenerativeModel("models/gemini-1.5-flash")

def ensure_ollama_running():
    pass  # No local check required for Gemini

def run_llm(prompt: str) -> str:
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        raise RuntimeError(f"[Gemini API Error] {e}")
```

PRACTICAL 11 (PRIVATA) · FILE 18 OF 23 **FORMAT: PYTHON · LENGTH: 26 LINES**

Privata Local Embedder (`privata/retriever/embedder.py`)

HuggingFace sentence-transformers/all-MiniLM-L6-v2 local embedder

```python
# privata/retriever/embedder.py



from typing import List
# from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_huggingface import HuggingFaceEmbeddings
from config import Config


# Initialize embedder once at module level
_embedder = HuggingFaceEmbeddings(
    model_name=Config.ingest["embedding_model"]
)

def embed_texts(texts: List[str]) -> List[List[float]]:
    """
    Embeds a list of text chunks and returns their vector representations.
    """
    return _embedder.embed_documents(texts)

def embed_query(query: str) -> List[float]:
    """
    Embeds a single user query.
    """
    return _embedder.embed_query(query)
```

PRACTICAL 11 (PRIVATA) · FILE 19 OF 23 **FORMAT: PYTHON · LENGTH: 54 LINES**

Privata ChromaDB Interface (`privata/retriever/vectordb.py`)

Persistent local vector store storage and cosine similarity retrieval

```python
# privata/retriever/vectordb.py

import os
from typing import List
from langchain_community.vectorstores import Chroma
from langchain.schema import Document
from config import Config
from retriever.embedder import _embedder


def get_vector_store() -> Chroma:
    """
    Initializes or loads the Chroma vector store.
    """
    persist_dir = Config.ingest["persist_directory"]
    collection = Config.ingest["collection_name"]

    return Chroma(
        collection_name=collection,
        embedding_function=_embedder,
        persist_directory=persist_dir
    )


def add_documents_to_store(store: Chroma, docs: List[Document]) -> None:
    """
    Adds documents to the Chroma vector store and persists them.
    """
    if not docs:
        print("[INFO] No documents to store.")
        return
    store.add_documents(docs)
    store.persist()


def retrieve_similar_docs(store: Chroma, query_vector: List[float], top_k: int) -> List[Document]:
    """
    Retrieves top-k most similar documents for a given query embedding.
    """
    return store.similarity_search_by_vector(query_vector, k=top_k)


def reset_vector_store():
    """
    Deletes the persisted Chroma DB for a clean reset.
    """
    persist_dir = Config.ingest["persist_directory"]
    if os.path.exists(persist_dir):
        for f in os.listdir(persist_dir):
            os.remove(os.path.join(persist_dir, f))
        print(f"[INFO] Vector store reset: {persist_dir}")
    else:
        print("[INFO] No vector store found to reset.")
```

PRACTICAL 11 (PRIVATA) · FILE 20 OF 23 **FORMAT: PYTHON · LENGTH: 56 LINES**

Privata Document Loaders (`privata/utils/doc_loader.py`)

Extractors for PDF, TXT, and Markdown files with metadata retention

```python
# privata/utils/doc_loader.py
# privata/utils/doc_loader.py

from pathlib import Path
from typing import List
from langchain.schema import Document
from langchain_community.document_loaders import (
    UnstructuredPDFLoader,
    TextLoader,
    UnstructuredMarkdownLoader,
)



def discover_files(input_path: str) -> List[Path]:
    """Recursively discovers supported document files."""
    path = Path(input_path)
    if not path.exists():
        raise FileNotFoundError(f"Input path not found: {input_path}")

    extensions = {".pdf", ".txt", ".md"}
    return [f for f in path.rglob("*") if f.suffix.lower() in extensions]


def load_file(path: Path) -> List[Document]:
    """Loads a single file using the appropriate loader."""
    loader_map = {
        ".pdf": UnstructuredPDFLoader,
        ".txt": TextLoader,
        ".md": UnstructuredMarkdownLoader,
    }

    loader_cls = loader_map.get(path.suffix.lower())
    if not loader_cls:
        print(f"[SKIP] Unsupported file: {path.name}")
        return []

    try:
        loader = loader_cls(str(path))
        docs = loader.load()
        for doc in docs:
            doc.metadata["source_file"] = str(path)
        return docs
    except Exception as e:
        print(f"[ERROR] Failed to load {path.name}: {e}")
        return []


def load_documents(input_path: str) -> List[Document]:
    """Loads and returns all documents from the given directory or file."""
    all_docs = []
    files = [Path(input_path)] if Path(input_path).is_file() else discover_files(input_path)
    for file in files:
        docs = load_file(file)
        all_docs.extend(docs)
    return all_docs
```

PRACTICAL 11 (PRIVATA) · FILE 21 OF 23 **FORMAT: TEXT · LENGTH: 22 LINES**

Privata Environment Requirements (`privata/requirements.txt`)

Python dependency manifest for local RAG execution

```
 1 # RAG Core
 2 langchain>=0.1.14
 3 langchain-community>=0.0.19
 4 langchain-chroma>=0.0.13
 5 sentence-transformers>=2.2.2
 6 unstructured[pdf]>=0.10.10
 7
 8 # Gemini API
 9 google-generativeai>=0.3.0
10
11 # System I/O
12 tqdm>=4.66.1
13 rich>=13.7.1
14 requests>=2.31.0
15 python-dotenv>=1.0.0
16
17 # Compatibility Fixes for Python 3.13
18 numpy<2.0.0
19 scikit-learn>=1.4.2
20 scipy>=1.12.0
21 pybind11>=2.12
22 transformers>=4.41.1
```

PRACTICAL 12 · FILE 22 OF 23 **FORMAT: PYTHON · LENGTH: 233 LINES**

Security Vulnerability & Mitigation Assistant (`p12_security_research_assistant.py`)

AI application for automated vulnerability triage, CWE classification, and defensive code remediation

```python
#!/usr/bin/env python3
"""
P12 — AI Security Vulnerability & Mitigation Assistant
Reuses Privata's RAG infrastructure (ChromaDB + MiniLM + Gemini API) for
automated vulnerability triage, root cause diagnosis, and defensive remediation.

Commands:
  cve <CVE-ID>        - Structured CVE vulnerability analysis and impact triage
  audit <CVE-ID>      - Defensive code audit pinpointing insecure query patterns
  mitigate <CVE-ID>   - Prescriptive defensive engineering and patch recommendations
  report <CVE-ID>     - Complete standardized Security Advisory & Remediation Report
"""

import os
import sys
import argparse
from pathlib import Path

# Add Privata to path
PRIVATA_ROOT = Path(__file__).parent / "privata"
sys.path.insert(0, str(PRIVATA_ROOT))

STUDENT_NAME = "Aayush Bankar"
STUDENT_ROLL = "246230316006"

# Curated knowledge base for vulnerability triage
MOCK_VULN_DB = {
    "CVE-2026-28576": {
        "title": "Android Contacts Provider SQL Injection",
        "cwe": "CWE-89: Improper Neutralization of Special Elements used in an SQL Command",
        "component": "ContactsProvider2 (com.android.providers.contacts)",
        "vector": "Local IPC via ContentResolver.query() selection parameter",
        "cvss": "7.5 / High (CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)",
        "impact": "Unauthorized extraction of contact records, call metadata, and phone identifiers",
        "root_cause": (
            "Legacy backward-compatibility mode: Applications targeting targetSdk < 30 "
            "bypass framework-level ENFORCE_STRICT_SQL_CHECKS validation. The underlying "
            "SQLiteQueryBuilder directly concatenates untrusted selection clauses into raw SQL queries."
        ),
        "vulnerable_pattern": (
            "// Insecure Selection Query Construction\n"
            "val rawSelection = \"account_name = '\" + userInput + \"'\"\n"
            "val cursor = contentResolver.query(\n"
            "    ContactsContract.RawContacts.CONTENT_URI,\n"
            "    projection, rawSelection, null, null\n"
            ")"
        ),
        "mitigation_code": (
            "// Secure Parameterized Query Construction\n"
            "val secureSelection = \"account_name = ?\"\n"
            "val selectionArgs = arrayOf(userInput)\n"
            "val cursor = contentResolver.query(\n"
            "    ContactsContract.RawContacts.CONTENT_URI,\n"
            "    projection, secureSelection, selectionArgs, null\n"
            ")"
        ),
        "defensive_steps": [
            "Mandate parameterized queries with bound selectionArgs across all ContentProvider endpoints.",
            "Modernize application manifests to targetSdk >= 31, activating mandatory platform SQL validation.",
            "Enable SQLiteQueryBuilder.setStrict(true) to restrict column projection and selection aliases.",
            "Implement fine-grained SELinux domain confinement and runtime permission checks (READ_CONTACTS)."
        ]
    }
}


def analyze_cve(cve_id: str) -> str:
    """Performs structured vulnerability analysis and impact triage."""
    if cve_id not in MOCK_VULN_DB:
        return f"[ASSISTANT] CVE {cve_id} not indexed in local knowledge base. Please import advisory."

    data = MOCK_VULN_DB[cve_id]
    return f"""# Vulnerability Triage Report: {cve_id}
Analyst: {STUDENT_NAME} (Roll: {STUDENT_ROLL}) | Dept. of Information Technology

1. Executive Summary & Classification
   - Identifier:       {cve_id}
   - Vulnerability:    {data['title']}
   - Weakness Class:   {data['cwe']}
   - Severity Score:   {data['cvss']}
   - Target Component: {data['component']}
```

```
82
83 2. Root Cause Analysis
84    {data['root_cause']}
85
86 3. Attack Vector & Scope
87    - Vector: {data['vector']}
88    - Impact: {data['impact']}
89
90 4. Remediation Baseline
91    - Parameterize selection parameters using bound arguments.
92    - Upgrade targetSdk to activate strict framework validation."""
93
94
95 def audit_vulnerability(cve_id: str) -> str:
96     """Performs static code audit demonstrating vulnerable vs secure patterns."""
97     if cve_id not in MOCK_VULN_DB:
98         return f"[ASSISTANT] No code audit rules found for {cve_id}."
99
100     data = MOCK_VULN_DB[cve_id]
101     return f"""# Security Code Audit: {cve_id}
102 Analyst: {STUDENT_NAME} (Roll: {STUDENT_ROLL})
103
104 [1] Flagged Insecure Implementation (String Concatenation):
105 ----------------------------------------------------------------------
106 {data['vulnerable_pattern']}
107 ----------------------------------------------------------------------
108 Risk: Untrusted input directly interpolated into SQL syntax without escaping.
109 Breaches: OWASP Mobile Top 10 (M4: Insufficient Input/Output Handling).
110
111 [2] Verified Remediation Implementation (Parameterized Query):
112 ----------------------------------------------------------------------
113 {data['mitigation_code']}
114 ----------------------------------------------------------------------
115 Verification: User input treated strictly as scalar data; prevents SQL syntax hijacking."""
116
117
118 def mitigate_cve(cve_id: str) -> str:
119     """Outlines defensive engineering mitigations and architectural hardening."""
120     if cve_id not in MOCK_VULN_DB:
121         return f"[ASSISTANT] Mitigation playbook unavailable for {cve_id}."
122
123     data = MOCK_VULN_DB[cve_id]
124     mitigations = "\n".join(f"   {i+1}. {step}" for i, step in enumerate(data['defensive_steps']))
125
126     return f"""# Defensive Engineering & Mitigation Playbook: {cve_id}
127 Analyst: {STUDENT_NAME} (Roll: {STUDENT_ROLL})
128
129 Key Remediation Countermeasures:
130 {mitigations}
131
132 Hardening Checklist:
133    [x] Static Code Analysis rule added to CI/CD pipeline
134    [x] targetSdk pinned to API level 31+
135    [x] Selection clause strictly bound with selectionArgs
136    [x] Zero plain text credentials or contact data exposed in logcat"""
137
138
139 def generate_report(cve_id: str) -> str:
140     """Generates a complete formal Security Advisory and Remediation Report."""
141     if cve_id not in MOCK_VULN_DB:
142         return f"[ASSISTANT] Cannot compile report: {cve_id} missing from advisory registry."
143
144     triage = analyze_cve(cve_id)
145     mitigate = mitigate_cve(cve_id)
146     return f"""======================================================================
147 FORMAL SECURITY ADVISORY & REMEDIATION REPORT
148 ======================================================================
149 {triage}
150
151 ----------------------------------------------------------------------
152 {mitigate}
153 ======================================================================
154 Report compiled for GTU Practical 12 Assessment.
155 """
156
157
158 def run_mock(args):
159     """Executes offline deterministic knowledge base queries."""
160     if args.command == "cve":
161         print(analyze_cve(args.target))
162     elif args.command == "audit":
163         print(audit_vulnerability(args.target))
164     elif args.command == "mitigate":
165         print(mitigate_cve(args.target))
166     elif args.command == "report":
167         print(generate_report(args.target))
168
169
170 def run_real(args):
```

```
    """Executes live RAG query via Privata's vector store and Gemini API."""
    try:
        from chat import start_chat_loop
        from retriever.vectordb import get_vector_store
        from retriever.embedder import embed_query
        from config import Config
        from models.llm_loader import run_llm
    except ImportError as e:
        print(f"[ERROR] Privata RAG modules not available: {e}")
        print("Falling back to local knowledge base execution...")
        run_mock(args)
        return

    Config.chat["prompt_template"] = """You are an AI Application Security Assistant specializing in defensive
software security, vulnerability analysis, and code hardening.

Context from Security Knowledge Base:
{context}

Question: {query}

Provide a structured, academic response containing:
1. Executive Summary & CWE Identification
2. Root Cause & Technical Impact
3. Defensive Mitigation & Code Hardening Strategy
4. Verification & Testing Checklist

Answer:"""

    print(f"[REAL] AI Security Assistant querying Privata knowledge store: {args.command} {args.target}")
    store = get_vector_store()
    query = f"Provide defensive security analysis, root cause, and mitigations for {args.target}"

    q_vec = embed_query(query)
    docs = store.similarity_search_by_vector(q_vec, k=Config.chat.get("top_k", 3))
    context = "\n---\n".join([d.page_content for d in docs])
    prompt = Config.chat["prompt_template"].format(context=context, query=query)
    response = run_llm(prompt)
    print(response)


def main():
    parser = argparse.ArgumentParser(
        description="AI Security Vulnerability & Mitigation Assistant (GTU Practical 12)"
    )
    parser.add_argument("--mock", action="store_true", default=True, help="Run offline knowledge base queries
(default)")
    parser.add_argument("command", choices=["cve", "audit", "mitigate", "report"], help="Security analysis command")
    parser.add_argument("target", help="CVE identifier (e.g. CVE-2026-28576)")
    args = parser.parse_args()

    print("=" * 70)
    print(f"AI SECURITY VULNERABILITY & MITIGATION ASSISTANT (mock={args.mock})")
    print(f"Command: {args.command} {args.target}")
    print(f"Student: {STUDENT_NAME} | Roll No: {STUDENT_ROLL} | Dept. of IT")
    print("=" * 70)

    if args.mock:
        run_mock(args)
    else:
        run_real(args)


if __name__ == "__main__":
    main()
```

PRACTICAL 12 · FILE 23 OF 23 **FORMAT: PYTHON · LENGTH: 179 LINES**

AI Study Assistant Prototype (`p12_study_assistant.py`)

Reference study assistant implementation for academic query answering

```python
"""P12 - AI Study Assistant (capstone, offline mock version).

An end-to-end AI application that combines:
  * a small knowledge base (study_notes) with retrieval,
  * a mock LLM client (or the live API client from P10 when AI_API_KEY is set),
  * three commands: explain / summary / quiz.

Design (see writeup for the full architecture diagram)
  prompt design  ->  retrieval (keyword + n-gram scoring)  ->  mock/real API

Run (offline):   python3 p12_study_assistant.py --mock
Run (live):      export AI_API_KEY=sk-...; python3 p12_study_assistant.py
"""

import argparse
import random
import re
import sys
from pathlib import Path

try:
    from p10_chatbot import ChatClient
except ImportError:  # allow running from a different cwd
    sys.path.insert(0, str(Path(__file__).parent))
    from p10_chatbot import ChatClient

NOTES = {
    "prompt engineering": (
        "Prompt engineering is the practice of designing instructions for an "
        "LLM to get accurate, well-formatted outputs. Key components: "
        "instruction, context, input data, output format. Core techniques: "
        "zero-shot, few-shot, role-based, chain-of-thought, prompt chaining.",
        "The quality of the prompt directly controls the quality of the answer.",
    ),
    "llm": (
        "A Large Language Model is a deep neural network trained on massive "
        "text corpora to predict the next token. It uses the transformer "
        "architecture with attention. LLMs power chatbots, summarizers, and "
        "code assistants. Known limits: hallucination, context window, bias.",
        "LLMs predict the next token based on the previous context.",
    ),
    "rag": (
        "Retrieval Augmented Generation grounds LLM answers in external "
        "documents. Pipeline: chunk the document, embed chunks as vectors, "
        "rank the chunks closest to the question, feed them to the LLM as "
        "context. RAG reduces hallucination and enables document Q&A.",
        "RAG = retrieve first, then generate with retrieved context.",
    ),
    "token": (
        "A token is a unit of text that an LLM reads and writes, typically a "
        "sub-word like 'prompt' or 'ing'. The context window is the maximum "
        "number of tokens a model can handle in one request. Longer prompts "
        "cost more because API billing is per token.",
        "Tokenization converts raw text into a sequence of tokens.",
    ),
    "agentic ai": (
        "An AI agent is a system that uses an LLM plus tools to act "
        "autonomously: it plans, calls tools (search, code, APIs), observes "
        "results, and iterates. Examples: AutoGPT, CrewAI. RAG adds knowledge; "
        "agents add action.",
        "Agents combine reasoning (LLM) with action (tools).",
    ),
}


def retrieve(topic: str, k: int = 2) -> list[str]:
    """Rank note entries by how many query words they share."""
    topic_words = set(re.findall(r"[a-z']+", topic.lower()))
    scored = []
    for key, (body, summary) in NOTES.items():
        hay = set(re.findall(r"[a-z']+", key + " " + body.lower()))
        scored.append((len(topic_words & hay), key, body, summary))
    scored.sort(key=lambda t: t[0], reverse=True)
    return [f"[{key}] {body} (one-liner: {summary})"
            for _, key, body, summary in scored[:k]]


def build_explain_prompt(topic: str, context: list[str]) -> str:
    return (
        "You are a friendly tutor for diploma IT students. Explain the topic "
        f"below in simple words in 3-4 sentences, with one real-world example.\n\n"
```

```
        f"TOPIC: {topic}\n\n"
        f"REFERENCE NOTES:\n" + "\n".join(context) + "\n\nEXPLANATION:"
    )


def build_quiz_prompt(topic: str) -> str:
    return (
        f"Generate 3 multiple-choice quiz questions (4 options each) about "
        f"'{topic}'. Give the correct answer after each question with a "
        "one-line reason."
    )


def mock_explain(topic: str, context: list[str]) -> str:
    top_key = context[0].split("]", 1)[0].strip("[]") if context else topic
    body = NOTES.get(top_key, (topic, ""))[0]
    first = body.split(".", 1)[0].lower()
    return (f"[mock] {topic.title()} in simple words: {first}. "
            f"It works best when you combine a clear definition with practice. "
            f"Example: ask your assistant 'explain {top_key}' and then quiz "
            "yourself on it.")


def mock_summary(topic: str, context: list[str]) -> str:
    top_key = context[0].split("]", 1)[0].strip("[]") if context else topic
    summary = NOTES.get(top_key, ("", ""))[1] or "No summary available."
    return f"[mock] Summary of '{top_key}': {summary}"


def mock_quiz(topic: str) -> str:
    questions = [
        ("Which technique shows a model examples before the real question?",
         "Few-shot prompting"),
        ("What does the context window limit?",
         "The number of tokens a model can process at once"),
        ("Which step comes first in a RAG pipeline?",
         "Chunking the document"),
    ]
    lines = []
    for i, (q, a) in enumerate(questions, 1):
        lines.append(f"{i}. {q}")
        for opt in (a, "Zero-shot prompting", "Fine-tuning", "Tokenization"):
            marker = " (correct)" if opt == a else ""
            lines.append(f"   - {opt}{marker}")
    lines.append("\n[Note: in live mode the model generates topic-specific MCQs.]")
    return "[mock] Quiz for you:\n" + "\n".join(lines)


def main() -> int:
    parser = argparse.ArgumentParser(description="AI Study Assistant (P12)")
    parser.add_argument("--mock", action="store_true",
                        help="use the offline mock brain (no API key)")
    parser.add_argument("--topic", default="prompt engineering",
                        help="topic to study")
    parser.add_argument("--provider", choices=["openai", "gemini"],
                        default="openai")
    args = parser.parse_args()

    try:
        client = ChatClient(provider=args.provider, mock=args.mock)
    except RuntimeError as e:
        print(f"[error] {e}", file=sys.stderr)
        return 1

    print("=" * 72)
    print(f"AI STUDY ASSISTANT (mock={args.mock}, provider={args.provider})")
    print("Commands: explain <topic> | summary <topic> | quiz <topic>")
    print("=" * 72)

    topic = args.topic
    context = retrieve(topic)
    print(f"\n[retrieval] top {len(context)} note(s) for '{topic}':")
    for c in context:
        print("  -", c[:100], "...")

    if args.mock:
        print("\n>>> explain")
        print(mock_explain(topic, context))
        print("\n>>> summary")
        print(mock_summary(topic, context))
        print("\n>>> quiz")
        print(mock_quiz(topic))
        return 0

    commands = [
        (f"explain {topic}", build_explain_prompt(topic, context)),
        (f"summary of {topic}",
         f"Summarize the key points of '{topic}' in exactly 3 bullets."),
        (f"quiz on {topic}", build_quiz_prompt(topic)),
```

```
    ]
    for name, prompt in commands:
        print(f"\n>>> {name}")
        print(client.respond(prompt))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```